

DNA algorithms for computing shortest paths

Ajit Narayanan and Spiridon Zorbalas

Department of Computer Science

University of Exeter

Exeter EX4 4PT

United Kingdom

ajit@dcs.ex.ac.uk

ABSTRACT

DNA computing has recently generated much interest as a result of pioneering work by Adleman and Lipton. Their DNA algorithms worked on graph representations but no indication was provided as to how information on the arcs between nodes on a graph could be handled. The aim of this paper is to extend the basic DNA algorithmic techniques of Adleman and Lipton by proposing a method for representing simple arc information — in this case, distances between cities in a simple map. It is also proposed that the real potential of DNA computing for solving computationally hard problems will only be realised when algorithmic steps which currently require manual intervention are replaced by executable DNA which operate on DNA strands in test-tubes.

1 Computationally hard problems and DNA computing

Both Adleman's (1994) and Lipton's (1995) algorithms deal with graphs where there are no labels on the arcs, nor is there any indication provided as to how such labels can be handled. The significance of their research is two-fold, however. They both demonstrate how DNA can be used for representing information (in the case of Adleman, representing graph nodes and their interconnectivity; in the case of Lipton, representing truth tables by converting a row of truth values into a graph and then adopting Adleman's representation technique. (See Narayanan (1997) for an introduction to these techniques.) Also, they both address problems in the complexity class NP

(in the case of Adleman, computing Hamiltonian Paths, and in the case of Lipton, solving SAT problems).

The *travelling salesperson problem* (TSP) to be tackled below is a variant of the *Hamiltonian Path Problem* (HPP): given a graph consisting of nodes (vertices) linked by edges (binary paths), find a route/path (concatenation of binary paths) which starts at a given node and ends at another given node, visiting every other node exactly once. The TSP is a variant of the HPP in that it asks for the *shortest* route/path between two given nodes, assuming that the binary paths are labelled with distances.

It has been conservatively estimated that to compute a shortest route, given a start and end node, which visits each city just once in a 30 city, fully interconnected network will take several million years, even assuming a billion instructions executed per second.

The point here is that there is no known algorithm which works in polynomial time for identifying a graph's Hamiltonian Paths and shortest routes between two nodes, and therefore the HPP and TSP are in the NP class of problems, meaning that *if* there were a non-deterministic machine which could explore each of the routes in parallel, each route can be computed in polynomial time, but the space of possible routes grows exponentially.

The difference between the two is that the HPP is NP-complete and the TSP, as defined above, is NP-hard. The HPP is known to fall in the class of problems where, if there is a solution to one of these problems, then there's a solution to all other problems in the class. If an algorithm returns a route, then it takes very little time to check that the route is correct. However, the TSP falls in the class of NP problems where to check that the answer (shortest route) is indeed correct may involve computing all routes to compare distances. The way to guarantee that the shortest route is indeed returned may not be applicable to other NP problems and therefore cannot be generalised to other intractable problems.

Two DNA algorithms for solving the TSP are presented below. The first only deals with distances on the binary paths, and the second, while also dealing with distances, can with suitable modifications and extensions also be used to deal with arc labels in general.

2 TSP DNA algorithm#1

Consider the simple map in Figure 2, which describes the binary paths and distances between four cities.

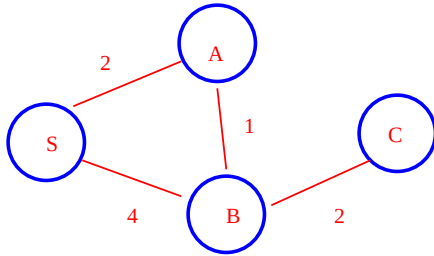


Figure 1: A simple map consisting of four cities and the distances between them.

The following sequence of steps, which are adapted from Adleman's DNA algorithm, will extract the shortest route between *S* and *C*.

1. Assign a unique DNA sequence for each city.
2. Construct DNA representations of binary paths between two cities *X* and *Y* as follows, where *n* is the distance (arc label) between the two cities (not necessarily symmetric):
 - (a) For the binary path $X \rightarrow Y$, join *n* occurrences of the DNA sequence for *X* with one occurrence of the DNA sequence for *Y*.
 - (b) For the binary path $Y \rightarrow X$, join *n* occurrences of the DNA sequence for *Y* with one occurrence of the DNA sequence for *X*.
 - (c) Form a complementary strand where the join takes place to hold the binary path together.
3. To form longer routes out of binary paths, *splice* two binary paths P_1 and P_2 together if and only if the final city code for P_1 matches the first city node for P_2 . Delete half of the code of the final city node in P_1 and half of the code of the first city in P_2 (which are matched).
4. To prevent loops, do not form longer routes if the final city in P_2 matches the first city in P_1 .
5. Repeat the above two steps until no more routes can be formed.
6. Place all the DNA sequences produced so far into a test tube.
7. Extract all those routes which start with the DNA code of the desired start city and place in a separate test tube.
8. Extract all those routes which end with the DNA code of the desired destination city and place in a separate test tube.

9. Sort all the remaining routes by length. Gel electrophoresis, whereby DNA molecules placed in an electric field migrate at different rates depending on their length towards the positive pole, can be used here. 'DNA molecules are negatively charged and under an electrical field DNA molecules migrate through the gel at rates dependent upon their sizes: a small DNA molecule can thread its way through the gel easily and hence migrates faster than a larger molecule' (Old and Primrose, 1985). The shortest DNA sequence represents the shortest route between the desired start and destination cities. Also, the sequence of DNA codes in the shortest strand provides information as to the order in which cities are encountered, and the length of the route can be calculated as follows: (total length of shortest strand / number of bases for each city) minus 1.

For example, the following DNA codes can be assigned to each city in Figure 2: $S = \text{AAAA}$; $A = \text{CCCC}$; $B = \text{GGGG}$; and $C = \text{TTTT}$ (Step 1). For the sake of exposition, clearly distinguishable DNA codes are used in this example for representing cities. The use of complementary codes for each city (i.e. CCCC is complementary to TTTT , and GGGG to CCCC) cannot be allowed in real DNA computation, since paths containing these complementary codes will be attracted to each other in test-tubes, resulting in badly formed DNA strands. Binary paths can then be formed, as described in top half of Figure 2. For instance, the binary path $S \rightarrow B$ has four occurrences of AAAA to represent the path length 4, followed by one occurrence of GGGG (Step 2). Longer routes can subsequently be formed also (lower half of Figure 2 — Step 3). For instance, the binary path $S \rightarrow B \rightarrow A$ is formed as follows:

$$\begin{array}{rcccl}
 & & \text{TTCC} & & \text{CCGG} \\
 \text{AAAAAAAAAAAAAAAAAGGGG} & + & & & \text{GGGGCCCC} = \\
 & & \text{TTCCCCGG} & & \\
 \text{AAAAAAAAAAAAAAAAAGGGGCCCC} & & & &
 \end{array}$$

where the last two bases of $S \rightarrow B$ and the first two bases of $B \rightarrow A$ are deleted because of the match.

The four longer paths in the lower half of Figure 2 are placed in a test tube (Step 6), and all those strands with the desired start and end points are extracted (Steps 7 and 8), in this case, the strands for $S \rightarrow B \rightarrow C$ and $S \rightarrow A \rightarrow B \rightarrow C$. (Different start and end points can be specified to extract other routes.) These two strands are then sorted to identify the shortest strand (Step 9), in this case $S \rightarrow A \rightarrow B \rightarrow C$, which is 24 bases long (as opposed to 28 for the other route). This strand contains within it the order in which the cities are to be visited (reading left to right), and total length can be calculated as the total number of bases (24) divided by the length of each city (4 bases), minus 1, resulting in route length 5.

As pointed out by Hartmanis (1995) and Amos *et al.* (1997), such algorithms don't realistically scale up from toy examples because of the huge amounts of DNA required to form the

S - A: TTGG
AAAAAAACCCC

A - S: GGTT
CCCCCCC AAAA

S - B: TTCC
AAAAAAAAAAAAAAAAAGGGG

B - S: CCTT
GGGGGGGGGGGGGGG AAAA

A - B: GGCC
CCCCGGGG

B - A: CCGG
GGGGCCCC

B - C: CCAA
GGGGGGGGTTTT

C - B: AACC
TTTTTTTGGGG

S - A - B: TTGGGGCC
AAAAAAACCCCGGGG

S - B - A: TTCCCGGGG
AAAAAAAAAAAAAAAAAGGGGCCCC

S - A - B - C: TTGGGGCC CCAA
AAAAAAACCCCGGGGGGGGTTTT

S - B - C: TTCC CCAA
AAAAAAAAAAAAAAAAAGGGGGGGGTTTT

Figure 2: The DNA sequences for binary paths and longer routes/paths using Algorithm #1.

initial set of routes. Hartmanis calculates that adopting Adleman's HPP DNA algorithm would require a mass of DNA greater than the Earth to solve a 200-city problem. Any DNA algorithm for the TSP which adds further complexity by requiring distances between nodes to be represented by multiple occurrences of DNA codes for nodes (as our algorithm above requires) will just compound the problem of scale. Also, Adleman (1994) himself points out the increasing error-proneness of DNA computations during ligation (joining), amplification (copying) and separation (sorting). Nucleotides (bases) degrade over time, and the more strands there are the more chance there is that the result of the DNA computation is not correct.

3 TSP DNA algorithm#2

A second algorithm is now proposed which attempts to bypass the above problems. Adleman's (1994) notation for representing strands and their complements is now used.

1. Let V be the total number of nodes in the graph and P the total number of binary paths in the graph. Sort the binary paths by distance, with the shortest binary paths occurring first, and place the binary paths in D .

2. Create the following strands:

- (a) O_i ($i = 1, \dots, V$) fixed length random sequences corresponding to all nodes (vertices) of the graph;
- (b) $\hat{O}_i$ ($i = 1, \dots, V$) fixed length sequences corresponding to the complements of O_i ;
- (c) O_d ($d = 1, \dots, P$) variable length random sequences corresponding to all the distances D in the graph;
- (d) $\hat{O}_d$ ($d = 1, \dots, P$) variable length sequences corresponding to the complements of O_d .

The lengths l of O_d and $\hat{O}_d$ will be proportional to the location of the corresponding distance in D and can, where appropriate, increase by a constant factor k : $\forall l \in 1, \dots, P \Rightarrow l = d * k$. (For example, if $k = 3$ and $D = \{2, 5, 9, 10\}$, then 2 is represented by a strand of length 3, 5 by a strand of length 6, 9 by a strand of length 9, and 10 by a strand of length 12.) (Problems associated with coding for distances in this way will be identified later.)

3. Let i be the start node of a path, d the path's distance, and j the end node of that path. We create strands representing every binary path between two nodes in the graph as follows:

- (a) **if $i = 1$ then**
create strand $O_{i-d} \rightarrow j$ as ALL O_i + ALL O_d + HL O_j ;
- (b) **if $i > 1$ then create strands**
 $O_{i-d} \rightarrow j$ as HR O_i + ALL O_d + HL O_j ;
 $O_{j-d} \rightarrow i$ as HR O_j + ALL O_d + HL O_i

where 'ALL' represents the whole DNA strand, 'HL' the left half part, 'HR' the right half part, and '+' is a join. Note that for every binary path in the graph except those emanating from the start node two strands are created, since (a) we are not looking for routes finishing at the start node and (b) every other route can be traversed in two (possibly non-symmetrical) ways.

4. Insert into a test-tube all $O_{i-d} \rightarrow j$ and $O_{j-d} \rightarrow i$ followed by all $\hat{O}$ strands (i.e. all $\hat{O}_i$, $\hat{O}_j$ and all $\hat{O}_d$), and perform a DNA ligase reaction in which random routes through the graph are formed. For instance, if $O_i = \text{TTAA}$ then $\hat{O}_i = \text{AATT}$; if $O_j = \text{ATAT}$ then $\hat{O}_j = \text{TATA}$; and if $O_d = \text{CCGGCC}$ then $\hat{O}_d = \text{GGCCGG}$. If $i = 1$ then the path $O_{i-d} \rightarrow j$ is **TTAACCGGCCAT** (ALL O_i + ALL O_d + HL O_j). If $i > 1$ then $O_{i-d} \rightarrow j$ is **AACCGGCCAT** (HR O_i + ALL O_d + HL O_j). When the $\hat{O}$ strands are added, we get:

AATTGGCCGGTATA
TTAACCGGCCAT

TTGGCCGGTATA
AACCGGCCAT

Note that the upper strand overshoots the lower strand by exactly half the length of a node, thereby allowing for paths starting with O_j to be concatenated to the lower strand through ligation, which in turn allows further $\hat{O}$ strands to be coupled.

5. Once the initial set of paths is formed, we amplify only those strands beginning with node 1 by a polymerase chain reaction using primers (Adleman, 1994; Boneh *et al.*, 1995) which specifically seek $\hat{O}_1$. Since $\hat{O}_1$ only occurs at the start of an upper strand, this effectively ensures that only those paths starting with the initial node are amplified.
6. When amplification terminates the test-tube will contain all routes starting with the initial node. Only those strands which terminate in the desired destination node are kept. These strands are then *melted* (Boneh *et al.*, 1995), i.e. double strands are separated into single strands through heating. The resulting single strands are *affinity-purified* (Adleman, 1994), a process whereby the strands are checked to ensure that they each contain each of the nodes in the network. This is achieved by successively generating each $\hat{O}_i$, $i = 1..V$ where V = the number of nodes in the network, and keeping only those strands to which each $\hat{O}_i$ binds at least and at most once.
7. All strands are then sorted in length through gel electrophoresis. The smallest strand contains the solution to the TSP.

An example of this algorithm solving the TSP for a simple map is provided in Figure 5. The problem with this algorithm is that proportional length coding of distances by some factor k is not guaranteed to return the correct result for all labelled graphs. Consider the following two sets of sorted distances: $D_1 = \{1, 2, 3, 4, 5\}$ and $D_2 = \{1, 12, 13, 14, 15\}$. Of the 10 possible concatenations of distances in D_1 (e.g. 1+2, 1+3, etc), five result in sums greater than the longest distance, 5. Of the 10 possible concatenations in D_2 , six result in sums greater than the longest distance, 15. If a constant increase in the length of distance DNA is adopted, there is a great danger that for some concatenations the combined, concatenated DNA of two distances, while less numerically than the longest distance, may be longer than the DNA for that longest distance, leading to errors in computing shortest paths. The obvious answer is to make the DNA length of a path directly proportional to the distance on that path. That is, for D_1 , a constant proportional increase of, say, 3 bases will indeed be fine, since the lengths all increase by 1. So the DNA code for 1+3, assuming 3 bases per unit length, will be 12 units long, which is still less than the DNA code for 5, which will be 15. For D_2 , however, a direct proportional representation will be needed, so that if each unit length is 3 bases long the DNA code for 1 will be 3 units long and the DNA code for 14 will be 42 units long (14×3) rather than 15 units long if proportional code is used.

4 Complexity aspects of TSP #2

The *quantity* of DNA can be estimated as follows, assuming constant proportional lengths of distances:

<i>Kinds of strand</i>	<i>Quantity</i>
O_i	n
$\hat{O}_i$	n
O_d	$\approx 2 * n * (n - 1)$ (worst case)
$\hat{O}_d$	$\approx 2 * n * (n - 1)$ (worst case)
Total	$2n * (2n - 1)$

This estimate does not take into account the need for multiple copies of strands to overcome errors. The *time complexity* can be roughly estimated as follows. These estimates are conservative and assume orders of complexity which are consistent with those adopted elsewhere in the literature.

<i>Operation</i>	<i>Procedure</i>	<i>Complexity</i>
<i>Sort</i>	sort distances	$O(t)$
<i>Anneal</i>	DNA ligase reaction	$O(1)$
<i>Polymerisation</i>	complement formation	$O(1)$
<i>Amplify</i>	PCR	$O(1)$
<i>Melt</i>	single strand generation	$O(1)$
<i>Extract</i>	affinity purification	$O(n)$
<i>Sort</i>	gel electrophoresis	$O(1)$
Total		$O(t + n + 5)$

The toy example discussed above uses simple distances, and further work is required to identify methods for representing more complex arc information, such as conditions to be satisfied before a transition can be made from one node to another (as, for example, in language transducers and augmented transition networks). Nevertheless, the example above demonstrates how arc labels can be represented by distinguishable DNA strands, of either constant proportion or of direct proportion.

5 The future for DNA computing

The major problem currently for Adleman's and Lipton's DNA computing experiments, and the ones described here, is the time involved in extracting and recombining DNA. While DNA processes within the test-tube can take place millions of times per second, extraction processes, whereby individual strands of DNA are manually isolated and spliced, can take several hours and even days, just for the simplest problems. This has led several researchers (e.g. Amos *et al.*, 1997) to conclude that the complexity aspects of DNA algorithms will limit their applicability. This conclusion, however, ignores some fundamental biological and computational issues. Current research in DNA computing uses DNA as a data structure ('representational DNA'), as for instance above, where DNA is used to represent a map. But any algorithm which only assumes manual manipulation of data representations is unlikely to fare well in terms of time taken to produce a result. Instead, the issue is whether all the steps involved in algorithms for

manipulating representational DNA can themselves be ‘automated’. Automated DNA can be achieved:

- by encoding certain algorithmic processes (those achieved through human intervention) as *executable DNA strands* which, when transcribed into messenger RNA and mapped onto enzymes within the test tube, can manipulate the representational DNA; and
- by introducing ready-made enzymes from outside to be combined in the same test tube as the representational DNA, so that operations currently executed manually outside the test tube take place within the test tube.

These two processes correspond, very roughly, to those cellular processes in which a cell’s DNA produces proteins and enzymes for use by itself (intracellular), and by other cells (extracellular), respectively. It is proposed that future DNA algorithms will need to appeal systematically to both sets of processes.

Here’s a simple example of how algorithmic processes can be represented as executable DNA, using a genetic code consisting of mapping *pairs* of bases into instructions. Real DNA is mapped three bases at a time onto amino acids (instructions). The example here is purely for exposition purposes. Consider the representational DNA strand **AGTGCTG** and the desired sequence of instructions on that strand below. These instructions are taken from Hofstadter’s (1979) system of Typogenetics — perhaps the first system to show how DNA could be used for representing data as well as for algorithm construction:

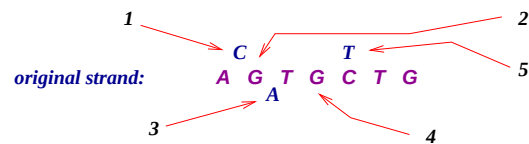
1. starting with leftmost unit in the representational strand, insert a **C** to the right of this unit;
2. search for the nearest purine (purines are **As** and **Gs**, pyrimidines are **Cs** and **Ts**) to the right;
3. insert an **A**;
4. search for the nearest purine to the right;
5. insert a **T** and finish.

The genetic code (using duplets rather than triplets) for these instructions is as follows:

GC insert a **C**
TC search for the nearest purine to the right
GA insert an **A**
GT insert a **T**

The sequence of instructions above can therefore be represented as **GC + TC + GA + TC + GT**, i.e. the executable DNA strand **GCTCGATCGT**. This DNA strand can be mapped onto an enzyme consisting of five amino acids (after transcription and translation within the virtual test tube), each of which individually executes one of the steps in the algorithm on a strand of representational DNA. For instance, if the representational DNA is **AGTGCTG**, the above executable DNA,

when mapped into the five steps and applied to this representational strand, produces **ACGATGTCTG**, as follows:



Step 1: insert a C to the right of the first unit

Step 2. Search for the nearest purine to the right (i.e. A or G)

Step 3. Insert an A to the right of this unit

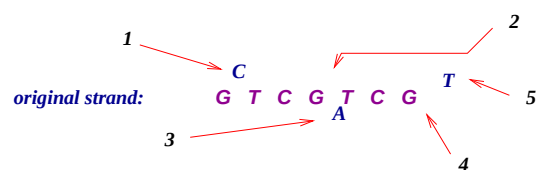
Step 4. Search for the nearest purine to the right

Step 5. Insert a T and finish

*i.e. **ACGATGTCTG***

Other possible instructions include cuts, searching for certain base sequences in either direction, forming complementary strands, and so on. There could be executable DNA for making copies of a representational DNA strand first before other executable DNA makes permanent changes to that strand, to ensure that original copies of representational DNA are kept for other executable DNA processes.

Using this approach, and given that transcriptase mechanisms and ribosomes can be made available in a virtual test tube to allow for the production of messenger RNA and the production of corresponding enzymes from the executable DNA, some surprisingly powerful mechanisms can be realised. For instance, assuming the same set of five executable instructions above and their executable DNA form **GCTCGATCGT**, but this time operating on the slightly different representational strand **GTCGTCTG**, we get as a result **GCTCGATCGT**, which is ... precisely the executable DNA sequence! A piece of data has been converted into an algorithm:



Step 1: insert a C to the right of the first unit

Step 2. Search for the nearest purine to the right (i.e. A or G)

Step 3. Insert an A to the right of this unit

Step 4. Search for the nearest purine to the right

Step 5. Insert a T and finish

*i.e. **GCTCGATCGT***

Another way to look at this is to say that the executable sequence has inserted itself into the representational DNA which, in turn, can find other representational **GTCGTCG** sequences to affect similarly. That is, the five instructions (amino acids) making up the enzyme act as a ‘virus’, given an appropriate strand, otherwise they result in a modified strand for other enzymes to work on. More interestingly, if such ‘viral’ processes can be implemented so that at each computational step two further copies of the algorithmic sequence can be generated which in turn find other representational DNA to transform, we have a mechanism for generating and manipulating an exponential search space in polynomial time because of the inherent parallelism involved. Other procedures for searching through this exponential space, also involving parallelism, may lead to novel, non-exponential methods for identifying the solution within the search space.

The above example demonstrates how executable instructions on representational DNA can be encoded with the DNA itself. Different techniques will be required when executable instructions are kept separate from representational DNA. In this case, the required instructions can be mapped onto enzymes outside the virtual test tube and the enzymes inserted (imported) into test tubes to carry out the manipulations required. This technique bypasses the need to introduce transcriptase and ribosome translation mechanisms into the test tube. It is expected that both techniques will have their uses, depending on the nature of the problem being tackled.

Proposals have already been made about basic DNA algorithmic operations (Boneh *et al.*, 1995): *extract* (extracting strands with given substrings); *length* (separating strands by length); *pour* (pouring the contents of two test tubes into one); *amplify* (making copies of strands or selected regions using polymerase chain reaction (PCR)); *anneal* (forming double strands out of single strands within a test tube); *cut* (cutting strands at specific points); and *join* (annealing the contents of two or more test tubes). Also, Rooss and Wagner (1996) identify 11 operations which they have added to Pascal (DNA-Pascal) in order to formalise basic DNA functions. Future research in DNA computing will no doubt evaluate the appropriateness of these operations for a variety of problems and to provide a methodology for taking a problem and giving it a DNA computational representation. However, it is proposed that the real potential of DNA computing will only become apparent when nearly all the steps which currently require manual implementation are themselves automated in test tubes, leading to significant speed increases (conservatively, by more than a trillion times, which is the rate of speed-up achieved by some enzymes in real biomolecular processes). Given the vast parallelism available in a test tube because of the size of DNA, it is possible that one test-tube of DNA, given the right instructions (including instructions for disassembling DNA strands which are not fruitful for a solution and recycling their constituents to other parts of the test tube, amplifying the most promising DNA strands first), can indeed solve, at molecular reaction speeds, problems which currently

take millions of years on conventional hardware.

Bibliography

Adleman, L. M. (1994). Molecular computation of solutions to combinatorial problems. *Science*, 266: 1021–1024.

Amos, M. Gibbons, A. and Dunne, P. E. (1997). The complexity and viability of DNA computations. In *Biocomputing and Emergent Computation*, D. Lundh, B. Olsson and A. Narayanan (Eds), World Scientific Press, pp 165–173.

Boneh, D., Dunworth, C., Lipton, R. J. and Sgall, J. (1995). On the computational power of DNA. Technical Report TR-499-95, Department of Computer Science, Princeton University. Available through <http://www.cs.princeton.edu/>.

Boneh, D. and Lipton, R. J. (1995). Making DNA computers error resistant. Technical Report TR-491-95, Department of Computer Science, Princeton University. Available through <http://www.cs.princeton.edu/>.

Hartmanis, J. (1995). On the weight of computations. *Bulletin of the European Association for Theoretical Computer Science*, 55: 136–138.

Hofstadter, D. R. (1979). *Gödel, Escher Bach: An Eternal Golden Brain*, Harvester Press.

Lipton, R. J. (1995). DNA solutions to hard computational problems. *Science*, 268 (28 April 1995): 542–545.

Narayanan, A. (1997). Representing arc labels in DNA algorithms. Research Report R360, Department of Computer Science, University of Exeter, Exeter EX4 4PT, UK. Available from ‘<http://www.dcs.ex.ac.uk/reports/reports.html>’.

Old, R. W. and Primrose, S. B. (1985). *Principles of Gene Manipulation* (3rd Edition). Blackwell Scientific.

Rooss, D. and Wagner, K. W. (1996). On the power of DNA computing. (Revised) Research Report RO-WAG96, available through <http://www.informatik.uni-wuerzburg.de/>. To appear in *Information and Computation*.

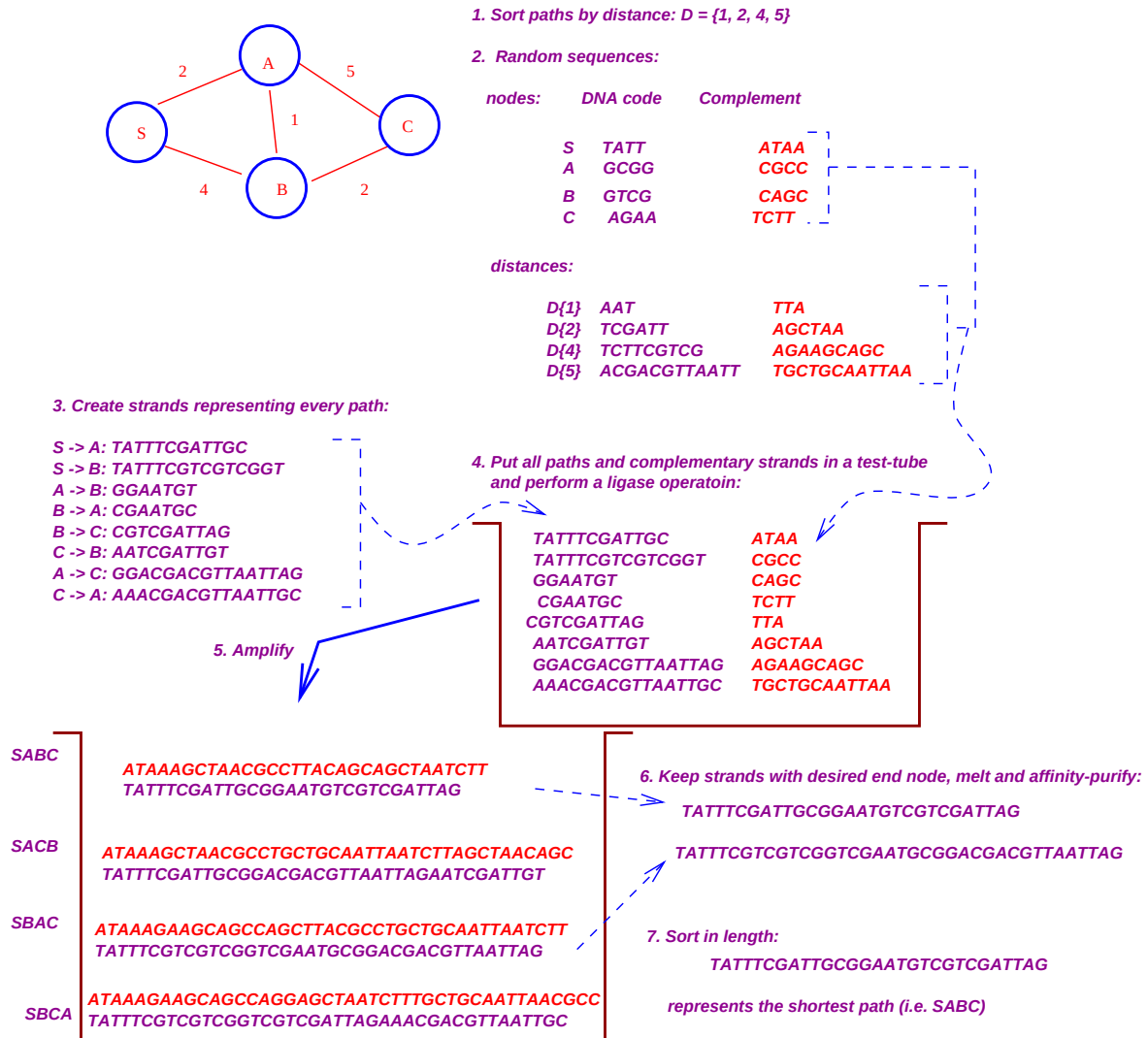


Figure 3: The seven steps of Algorithm #2. Various aspects of the algorithm are simplified for exposition purposes. When the strands are placed in the test-tube and amplified, only four relevant strands are shown (with the routes they represent labeled on the left outside the test-tube).